

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра инженерной психологии и эргономики

Специальность «Программная инженерия»

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №5
«Файловое хранилище»

Подготовил:

Козинцев М.Л.

Проверил:

Болтак С. В.

Минск 2026

Задание:

Необходимо реализовать удаленное файловое хранилище, предоставляющее REST API по протоколу HTTP. При реализации разрешается использовать веб-фреймворк, реализующий HTTP-роутинг и работу с HTTP-вызовами.

Путь URL запроса определяет логическое расположение файла в хранилище:

`http://storage.com/path/to/file.txt` - описывает файл `file.txt`, находящийся по пути `/path/to`.

Список поддерживаемых функций:

- загрузка файла в хранилище с перезаписью с помощью метода PUT;
- получение файла из хранилища с помощью метода GET;
- получение списка файлов каталога с помощью метода GET (в качестве формата данных для ответа рекомендуется использовать JSON);
- получение информации о файле (размер в байтах и дата последнего изменения) в виде HTTP-заголовков без получения содержимого файла с помощью метода HEAD;
- удаление файла/каталога из хранилища с помощью метода DELETE.

Необходимо корректно использовать коды состояния HTTP.

Для тестирования созданной службы использовать утилиту `curl` или аналоги (Postman, HTTPie, etc). Получение файла или содержимого каталога также должно быть возможным через браузер.

Исходные данные:

Фильтр в Wireshark: `tcp.port == 7099 || http`

На рисунке 1 представлено расположение и содержимое папки, представляющей собой хранилище.

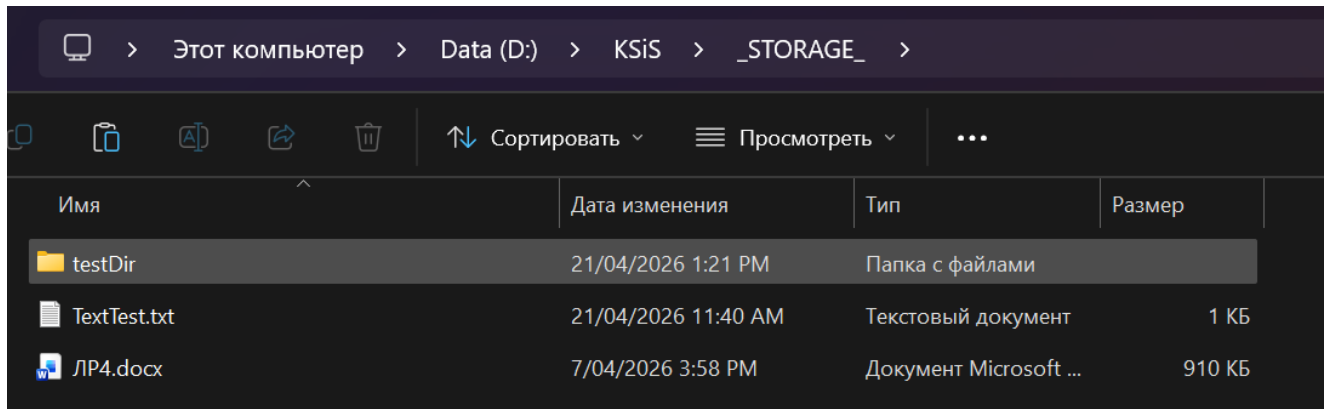


Рисунок 1 – Путь и содержимое хранилища

На рисунке 2 представлен запуск хранилища.

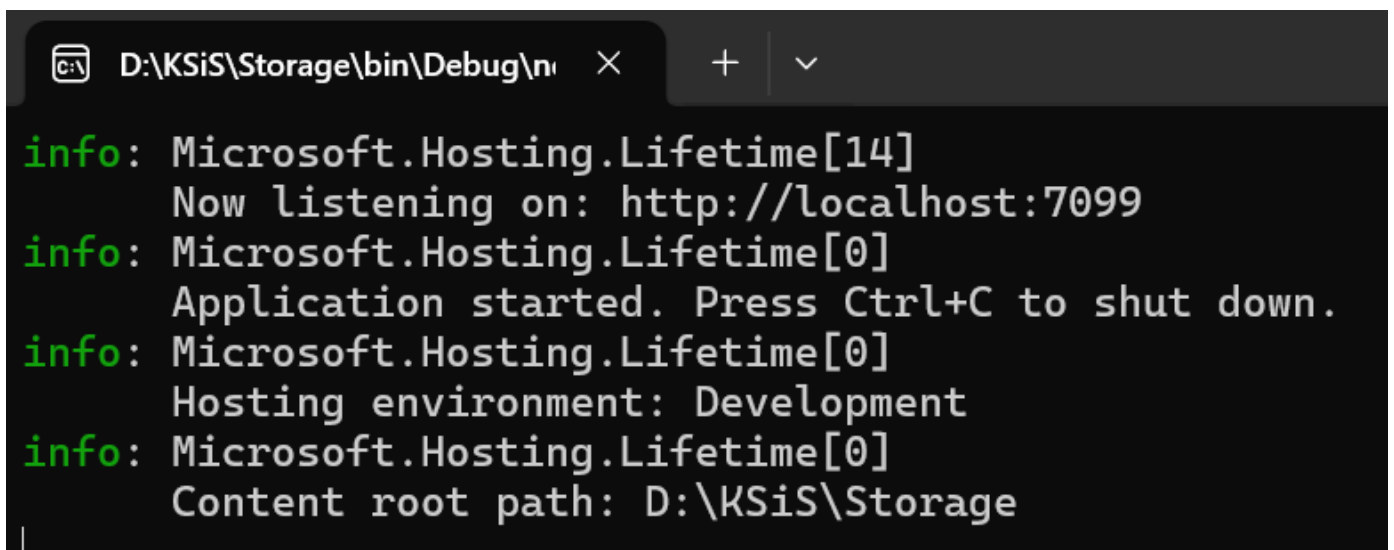
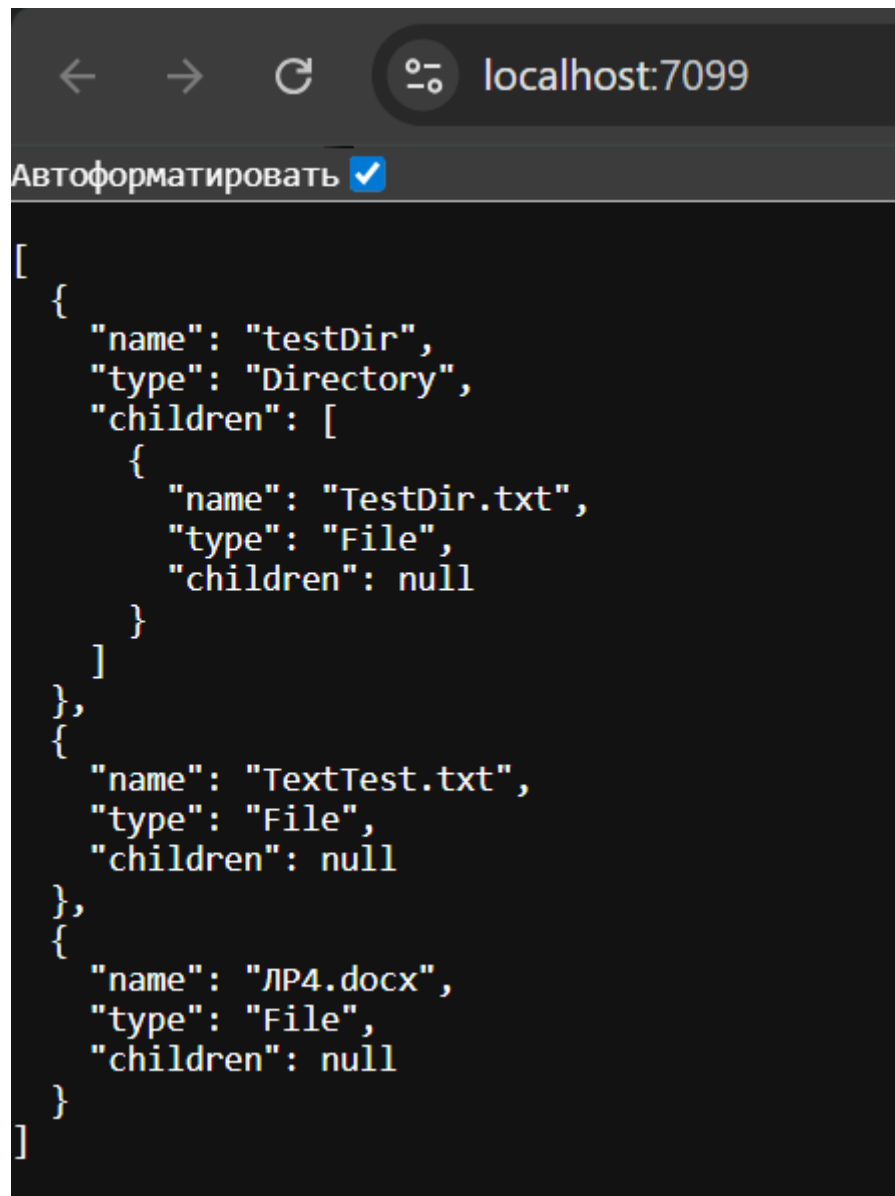


Рисунок 2 – Запуск и логирование сервера файлового хранилища

На рисунке 3 представлено получение списка файлов и директорий хранилища в браузере в JSON формате.



```
[
  {
    "name": "testDir",
    "type": "Directory",
    "children": [
      {
        "name": "TestDir.txt",
        "type": "File",
        "children": null
      }
    ]
  },
  {
    "name": "TextTest.txt",
    "type": "File",
    "children": null
  },
  {
    "name": "ЛР4.docx",
    "type": "File",
    "children": null
  }
]
```

Рисунок 3 – Содержимое хранилища

На рисунках 4-6 продемонстрировано получение файла из хранилища через браузер и через Postman с помощью метода GET с демонстрацией перехваченного трафика в Wireshark.

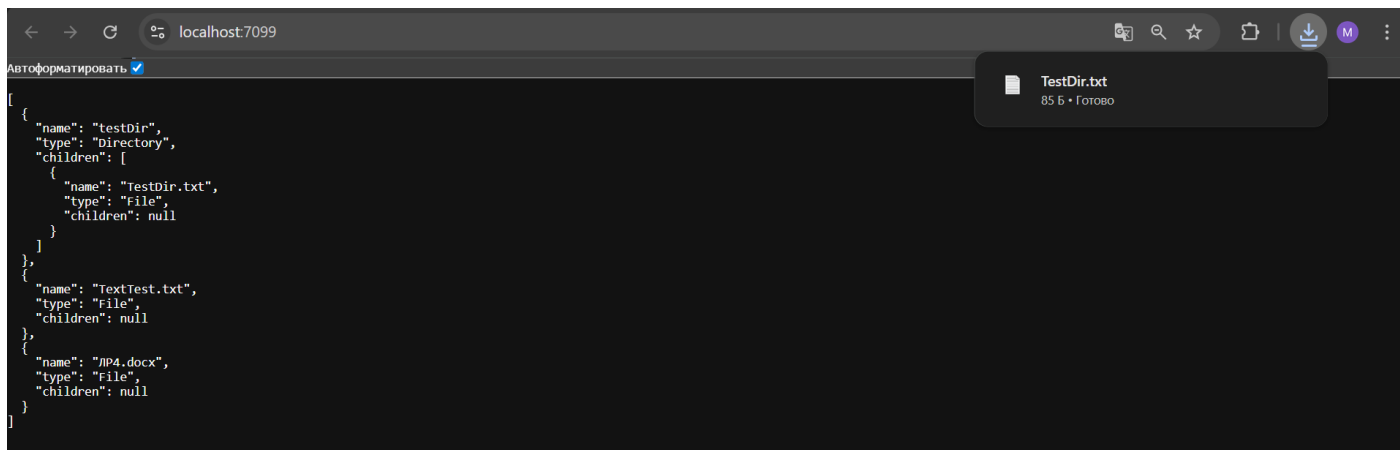


Рисунок 4 – Получение файла по пути testDir/TestDir.txt через браузер

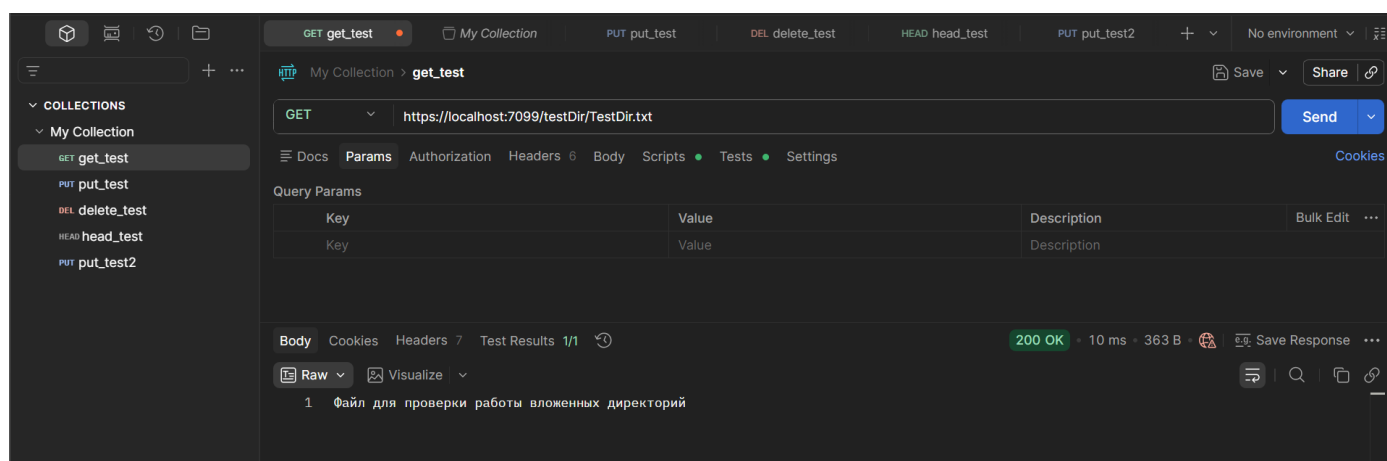


Рисунок 5 – Получение файла через Postman

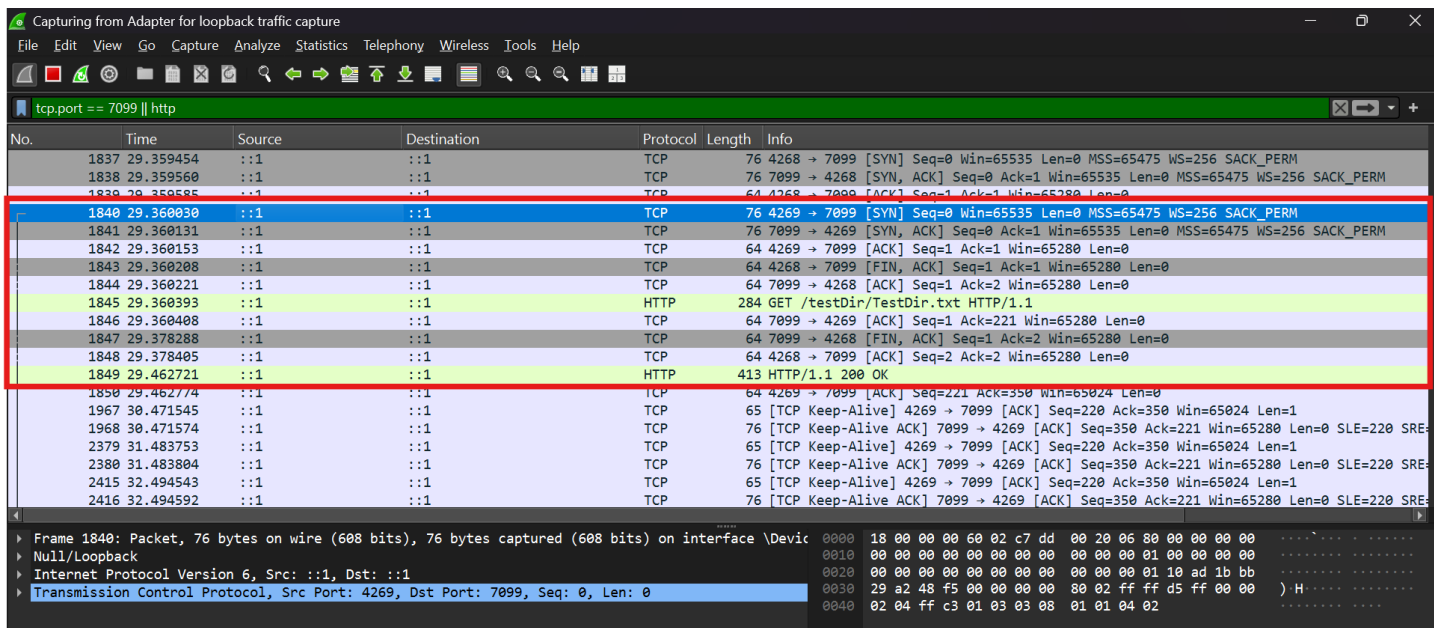


Рисунок 6 – Трафик в Wireshark с http-запросом с методом GET и ответом с кодом 200

На рисунках 7-8 представлено удаление файла TextTest.txt с помощью метода DELETE через Postman и подтверждающий это трафик в Wireshark.

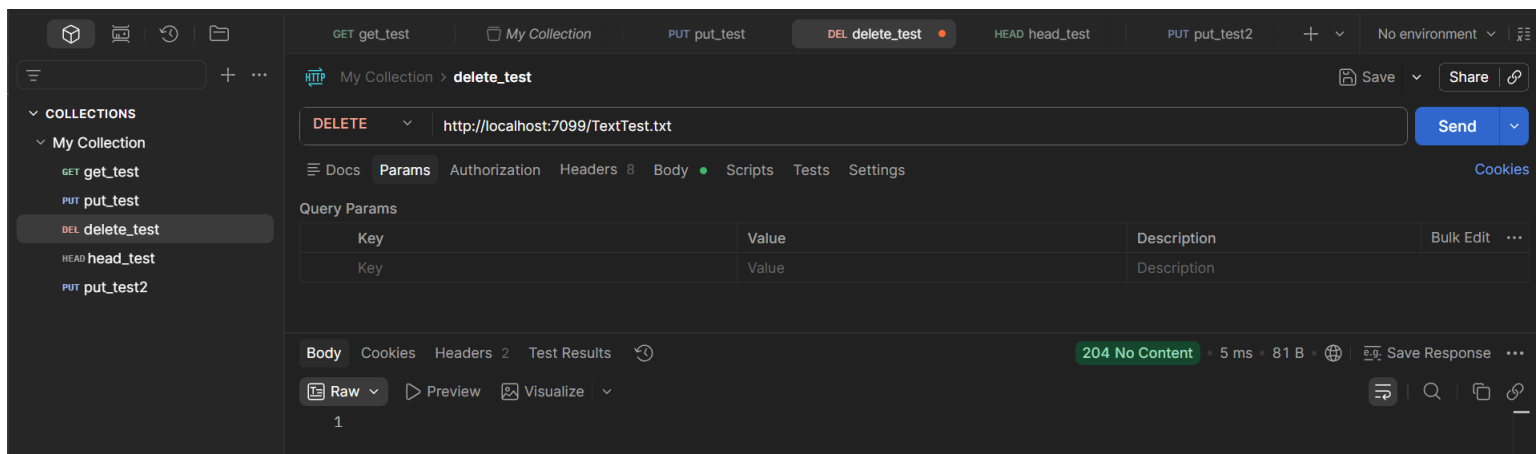
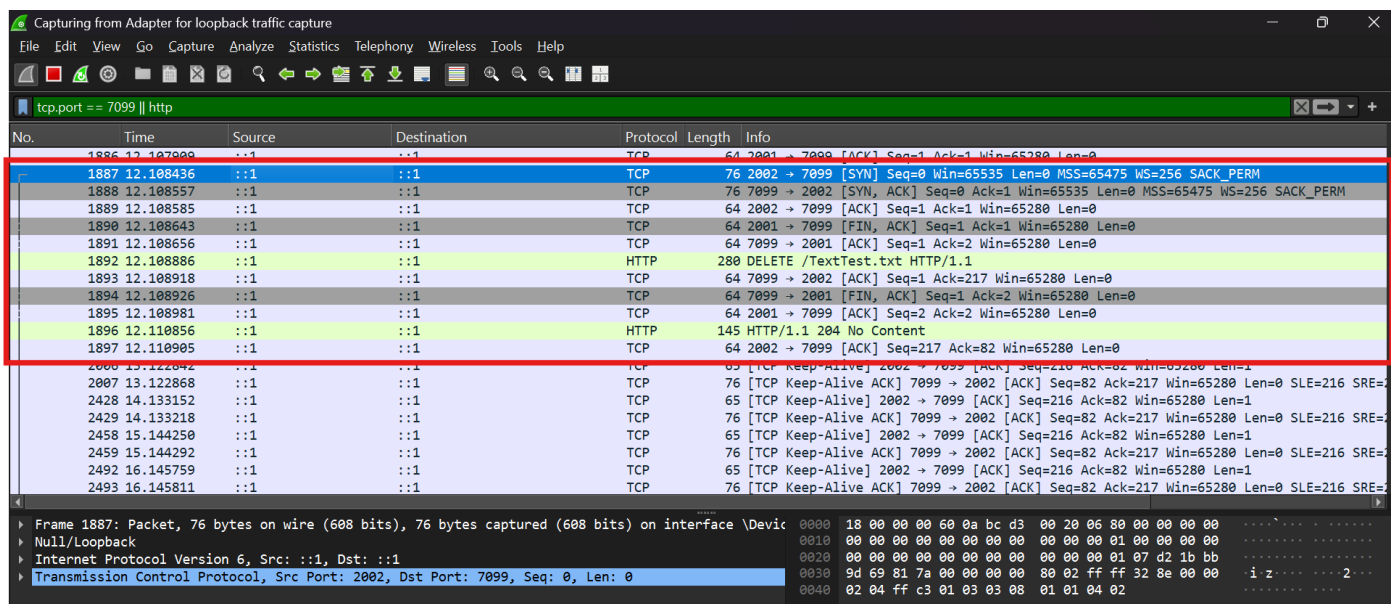


Рисунок 7 – Удаление файла через Postman



Рисунки 8 – Трафик в Wireshark с http-запросом с методом DELETE и ответом с кодом 204

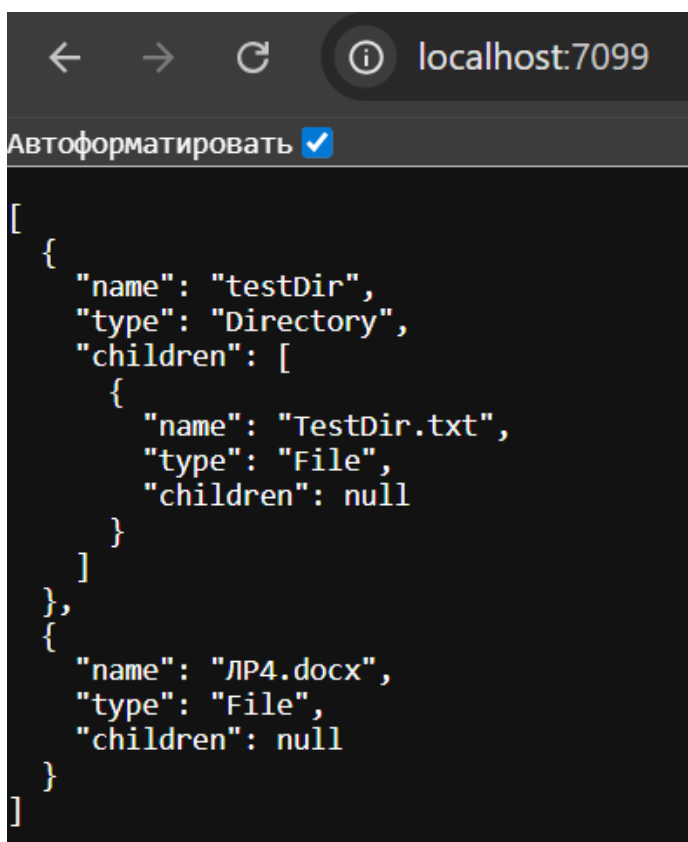


Рисунок 9 – Содержимое хранилища после удаления

На рисунках 10-11 представлена загрузка файла image.jpg в хранилище с помощью метода PUT через Postman и трафик в Wireshark.

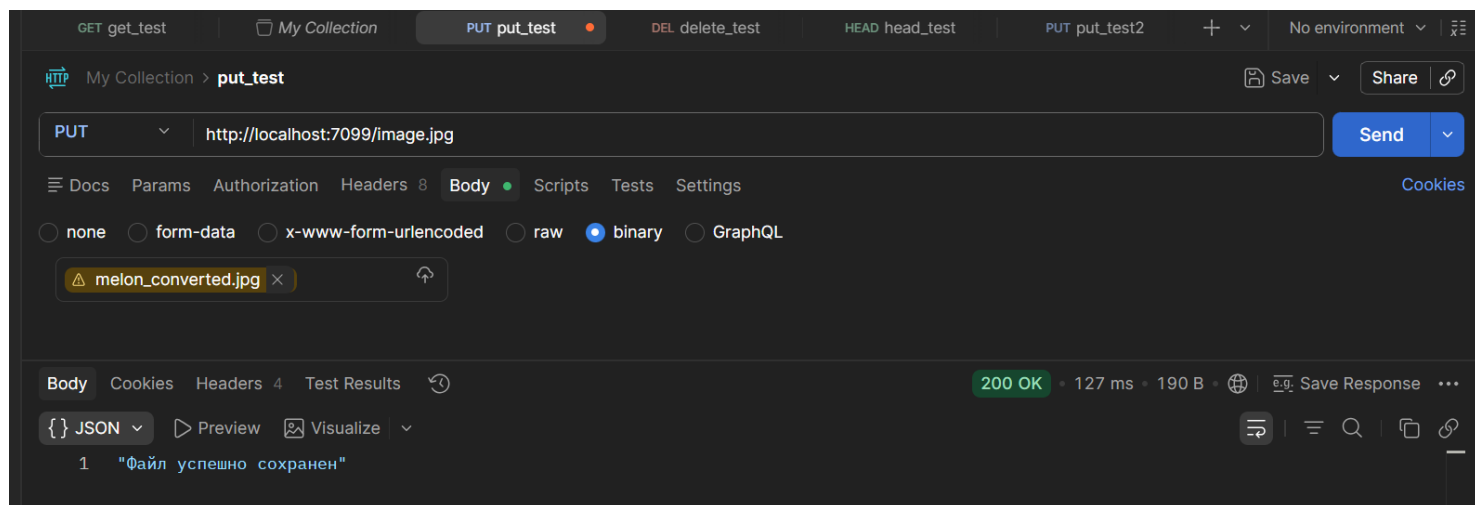


Рисунок 10 – добавление файла через Postman

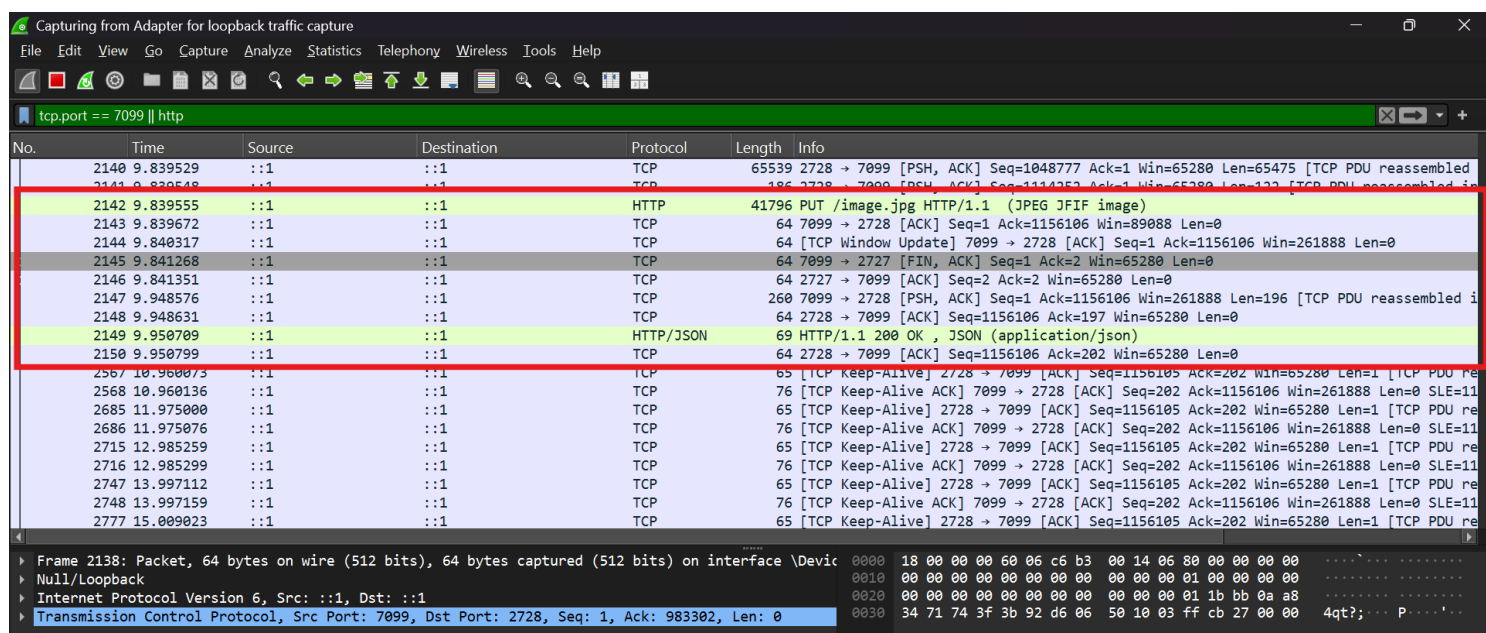


Рисунок 11 – Трафик в Wireshark с http-запросом с методом PUT и ответом с кодом 200



Рисунок 12 – содержимое хранилища поле добавления файла

На рисунках 13-14 представлен процесс получения информации о файле TestDir.txt с помощью метода HEAD через Postman и трафик в Wireshark.

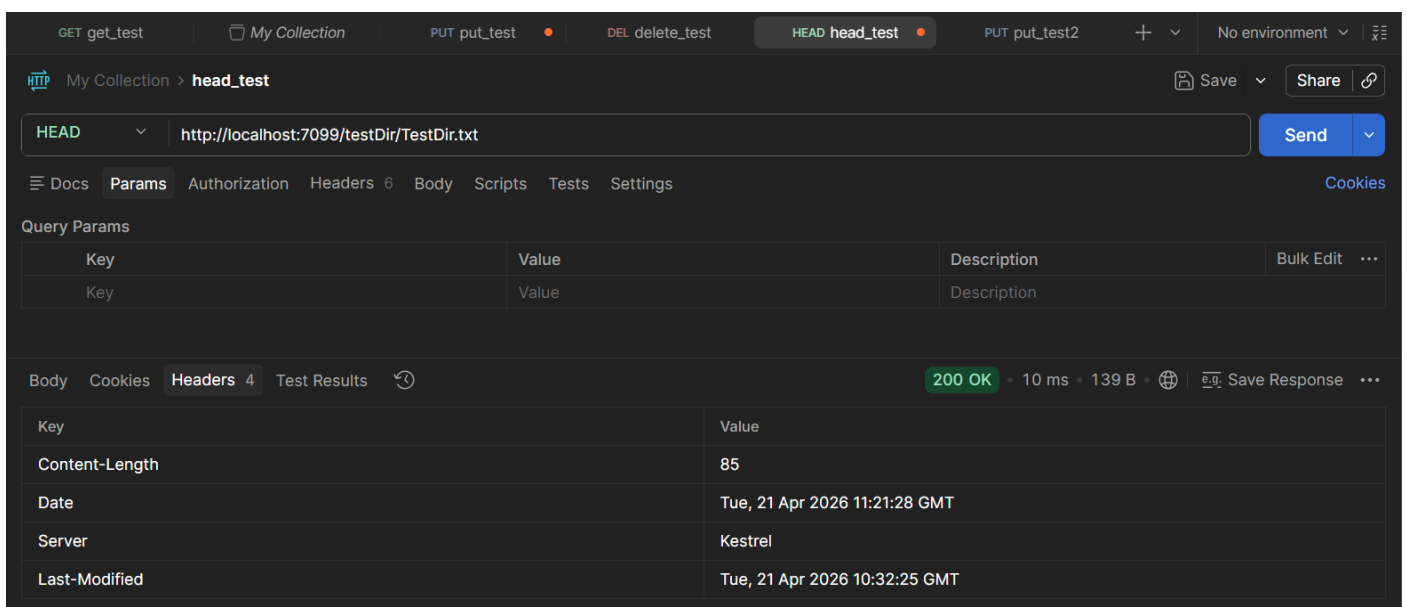


Рисунок 13 – Информация о файле TestDir.txt

Capturing from Adapter for loopback traffic capture

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port == 7099 || http

No.	Time	Source	Destination	Protocol	Length	Info
1870	11.405832	::1	::1	TCP	76	1684 → 7099 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
1871	11.405929	::1	::1	TCP	76	7099 → 1684 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
1872	11.405960	::1	::1	TCP	64	1684 → 7099 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1873	11.406374	::1	::1	TCP	76	1685 → 7099 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
1874	11.406455	::1	::1	TCP	76	7099 → 1685 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
1875	11.406476	::1	::1	TCP	64	1685 → 7099 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1876	11.406521	::1	::1	TCP	64	1684 → 7099 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
1877	11.406531	::1	::1	TCP	64	7099 → 1684 [ACK] Seq=1 Ack=2 Win=65280 Len=0
1878	11.406666	::1	::1	HTTP	285	HEAD /testDir/TestDir.txt HTTP/1.1
1879	11.406667	::1	::1	TCP	64	7099 → 1684 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
1880	11.406683	::1	::1	TCP	64	7099 → 1685 [ACK] Seq=1 Ack=222 Win=65280 Len=0
1881	11.406718	::1	::1	TCP	64	1684 → 7099 [ACK] Seq=2 Ack=2 Win=65280 Len=0
1882	11.413686	::1	::1	HTTP	203	HTTP/1.1 200 OK
1883	11.413730	::1	::1	TCP	64	1685 → 7099 [ACK] Seq=222 Ack=140 Win=65280 Len=0
1914	12.427659	::1	::1	TCP	65	[TCP Keep-Alive] 1685 → 7099 [ACK] Seq=221 Ack=140 Win=65280 Len=1
1915	12.427694	::1	::1	TCP	76	[TCP Keep-Alive ACK] 7099 → 1685 [ACK] Seq=140 Ack=222 Win=65280 Len=0 SLE=221 SRE=
2028	13.427977	::1	::1	TCP	65	[TCP Keep-Alive] 1685 → 7099 [ACK] Seq=221 Ack=140 Win=65280 Len=1
2030	13.428012	::1	::1	TCP	76	[TCP Keep-Alive ACK] 7099 → 1685 [ACK] Seq=140 Ack=222 Win=65280 Len=0 SLE=221 SRE=
2140	14.435632	::1	::1	TCP	65	[TCP Keep-Alive] 1685 → 7099 [ACK] Seq=221 Ack=140 Win=65280 Len=1
2141	14.435663	::1	::1	TCP	76	[TCP Keep-Alive ACK] 7099 → 1685 [ACK] Seq=140 Ack=222 Win=65280 Len=0 SLE=221 SRE=

Frame 1873: Packet, 76 bytes on wire (608 bits), 76 bytes captured (608 bits) on interface \Device\NPF{...} Null/Loopback

Internet Protocol Version 6, Src: ::1, Dst: ::1

Transmission Control Protocol, Src Port: 1685, Dst Port: 7099, Seq: 0, Len: 0

Рисунок 14 – Трафик в Wireshark с http-запросом с методом HEAD и ответом с кодом 200

Вывод: в ходе выполнения лабораторной работы было реализовано удаленное файловое хранилище на базе ASP.NET Core, обеспечивающее взаимодействие по протоколу HTTP с использованием архитектурного стиля REST. В процессе разработки были успешно настроены и протестированы методы GET, PUT, HEAD и DELETE. Реализация рекурсивного формирования древовидной структуры каталогов обеспечила наглядное представление иерархии файлов в формате JSON при запросах через веб-браузер. Применение асинхронных методов обработки потоков данных позволило обеспечить корректную передачу файлов в хранилище, итоговая проверка через Postman подтвердила стабильность работы API, правильность обработки HTTP-заголовков и соблюдение стандартов кодов состояния при различных сценариях обращения к ресурсам.